Distributed Perception for Drone Fleets: Federated Fine-Tuning of an Edge Vision-Language Decision Model for Inspection Task Planning

Mahdi Khashan

[a][Affiliation to be added],

Abstract

A fleet of inspection drones must decide on board, for every image, what it sees and what to do next: continue, escalate the frame, or revisit the asset. Generative vision-language models (VLMs) give this perception semantic depth but need seconds per image on a drone computer, and the images that would improve them stay spread across the fleet. We study a vision-language decision model, laya-vision (201M parameters), which answers typed inspection queries in one forward pass with calibrated probabilities that a task planner can consume directly, and we train it across the fleet with federated learning. On 35,415 power-line component images (ElecVQA-Bench, InsPLAD), the published checkpoint fails zero-shot (condition macro F1 0.180). Fine-tuning on folder labels raises it to 0.703 (defect AUROC 0.965) at 74 ms per image for four queries on an RTX 2080 Ti; as an escalation trigger, the defect probability forwards 15.3% of images and finds 88.5% of defects. With FedAvg over simulated tower-wise drones, macro F1 falls from 0.697 to 0.474 as the data is spread over 1 to 18 drones, but stays above drones that train alone (0.371). Across 25 federated configurations from 1,014 to 41,310 MB of total uplink, the starting model matters more than the uplink: from the fine-tuned start, a 45 MB update reaches 92% of the full-model fleet result at 20% of its uplink, while from the published checkpoint small updates barely learn. With few labels, fleet training pays off only once the fleet has labelled about 70 defects.

Keywords: drone fleets, distributed intelligence, edge AI, UAV perception, task planning, vision-language model, federated learning, infrastructure inspection

1. Introduction

Infrastructure inspection is moving from single drones that record video for later analysis to fleets of drones that interpret what they see on board and act on it [1]. Each drone of such a fleet runs a perception–planning loop: it perceives a component, judges its condition, and the mission planner decides whether to continue, to escalate the frame to a slower model or to the operator, or to schedule a closer look. The planner needs from perception not a caption but a few typed answers with probabilities it can trust.

Farahani et al. [1] show on a Jetson Orin that a lightweight VLM can run on the drone, but slowly: a fine-tuned SmolVLM-256M [4] needs about 2.8 s per image, and EVLM [2] 1.4–6.4 s per 4 s segment. Their agenda leaves open low-latency reasoning, an inexpensive stage that decides when the VLM runs, training data without GPT-4o captions, confidence a mission planner can act on, and collaborative training across UAVs.

Many inspection queries are choices among known options: “Which component is this?”, “Does it have a defect?”. Decision models answer such typed queries in one forward pass with a probability per option; laya-vision [6] brings this design [5] to images. Its output is directly an input for task planning. The images that would improve it are spread across the fleet, may belong to different operators and are often sensitive, which makes federated learning [9] the natural form of distributed intelligence for such a fleet, although each drone flies its own towers and therefore sees different data, and every model update competes with mission traffic for the uplink.

We ask how well a decision model perceives power-line components and their condition, zero-shot and after fine-tuning on labels that need no caption generation (RQ1); how fast it is and whether its calibrated output can drive the escalation decision of a planner (RQ2); how close federated fine-tuning across tower-wise drones comes to central training, and whether a centrally fine-tuned start helps the fleet (RQ3); and how much uplink the fleet needs, and what few labels per drone and drones with different image types do to the result (RQ4).

We contribute an evaluation of the decision model as the perception stage of an inspection drone on 35,415 InsPLAD images, a latency and escalation analysis, and a federated study over 25 configurations that relates the accuracy of the fleet to its uplink.

Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

2. Related work

Edge AI for drone perception. EVLM [2] runs a LoRA-tuned SmolVLM-256M on a Jetson for power-line inspection and cuts the uplink of a 4 s segment from 485 kB to 25 kB at 1.4–6.4 s of generation per segment; [3] profiles whether such a model should run on the drone, at the edge or split between them, and INAR-VL [7] routes requests between edge and cloud. All of these generate text. Laya [5] instead scores typed questions in one pass with a temperature per question type [8], and laya-vision [6] adds image input; we use its checkpoint and fine-tuning recipe.

Distributed intelligence and federated learning. FedAvg [9] averages client models weighted by data size; non-IID data degrades it [10], and FedProx [11] adds a proximal term. Federated learning has been applied to routing language models [13], to escalation thresholds [14], and to adapter updates in guardrail inspection [15]. We are not aware of federated fine-tuning of a vision-language decision model across an inspection fleet, or of a study that relates its accuracy to the uplink it needs.

3. System and method

3.1. Fleet model

A fleet of K drones inspects a power line; the mission planner assigns each drone a set of towers. On each image, the drone's decision model answers typed queries. The defect probability drives the escalation decision (forward the frame to a describing VLM or to the operator, or continue), and the component and condition answers feed the planner's record of the inspected assets. A server holds an initial model and, optionally, labelled images from earlier missions. Between missions, the drones fine-tune the model on images they have labelled and send model updates, never images, to the server, which aggregates them with FedAvg. Its cost is the uplink, update size × drones × rounds in MB, which we calculate; the drones are simulated on one GPU.

3.2. Model and queries

Laya-vision [6] (checkpoint thaitea/laya-vision, revision f2fe3c1, 201M parameters) uses SmolVLM-256M cut to 20 of its 30 language layers, followed by a decision head. For an image and a typed query, the head returns one logit per option; a temperature per query type converts them to probabilities. The model can only answer with one of the given options. One predict call per image encodes the image once and asks four queries:

- component (choice, 11 options): "Which power line component is in this image?"
- defect (yes/no): "Does this power line component have a defect?"
- condition (choice, 6 options): "Which condition does this power line component have?" with options good, rust, corrosion, missing cap, bird nest, or other defect (torn part or peeling paint).
- judgeable (yes/no): "Is the image clear enough to judge the condition of the component?" This query has no ground truth.

Figure 1 shows one test image of each condition with the answers of the fine-tuned model.

3.3. Data and ground truth

ElecVQA-Bench [16] builds visual questions on the InsPLAD component crops [17]. Power-line inspection is one of the four domains of [1], and its anomaly types (degradation, missing component, corrosion) are the classes of this data. We use the tower14 split: 18 training, 10 validation and 14 test towers, with 39,228 items on 35,415 unique images. The towers in different splits share no images. Each image carries its component folder and its class folder. We take the component (17 folders, with the 7 shackle types merged into one, giving 11 classes) and the condition (6 classes, with torn-up and peeling-paint merged) from these folders. A defect is any condition other than good. The folder labels agree with every ElecVQA-Bench answer. The test towers hold 6,092 images, of which 601 (9.9%) show a defect. No caption is generated at any step, so the training labels need no teacher model and no manual text annotation.

3.4. Fine-tuning

We follow the fine-tuning guide of laya-vision [6]. Each image gives three labelled questions (component, defect, condition). Training uses the RLCD objective of Laya with soft cross-entropy, AdamW, batch 8, learning rates 10^{-4} for the head and 2×10^{-5} for the backbone, 25 warm-up steps and 2.5 passes over the training questions. Temperatures are refit on held-out training images after training [8]. The RTX 2080 Ti has no native bfloat16, so all training runs in fp32. We train three models:

- Round 1: up to 400 images per condition (1,893 images, 79% defects; 1,793 for training), training the head and the last 4 language layers (22.6M parameters).
- Round 2, last 4 layers: all 2,249 defect images of the training towers plus up to 300 good images per component (5,107 images, 44% defects; 4,807 for training, 300 for temperatures).
- Round 2, full: the same data, training all parameters except the vision encoder (114.7M parameters).

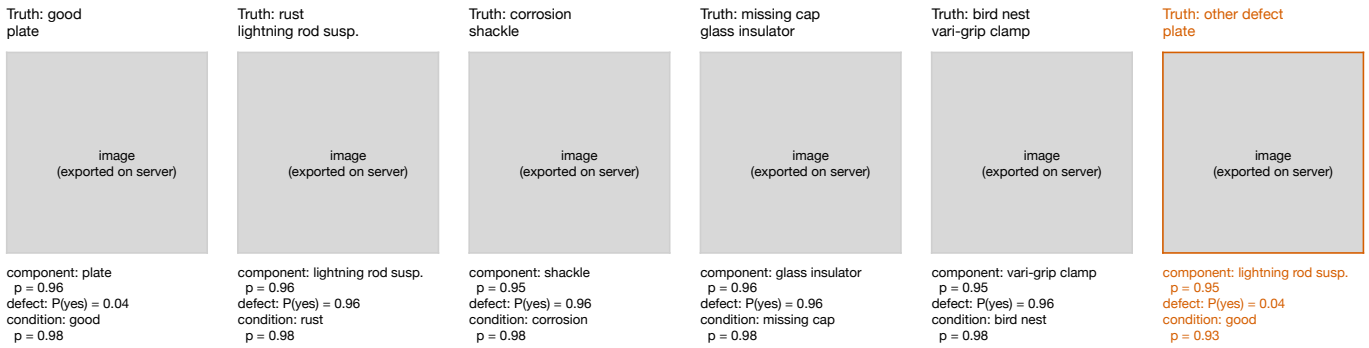


Figure 1: One query, three typed answers. One test-tower image of each condition, with the ground truth (top) and the answers of the round-2 full model to the component, defect and condition queries of one predict call (bottom; p is the probability of the chosen option). The first five are answered correctly; the last, a plate with an "other defect", is called a good lightning rod suspension; all five test images of this class are missed.

3.5. Federated fine-tuning

The simulated drones use the round-2 images, divided by tower with balanced image counts; with $K = 18$ each drone holds one tower (1 to 1,103 images, four drones without any defect). In round r, the server sends the global weights θ^r to every drone. Drone k trains on its n_k questions for 0.25 passes with a new AdamW optimizer and returns θ_k^r. The server averages

$$\theta^{r+1} = \sum_{k=1}^{K} \frac{n_k}{n} \theta_k^r, \qquad n = \sum_k n_k. \tag{1}$$

Ten rounds give the same 2.5 passes as central training, with a cosine learning-rate decrease over the rounds. Each drone keeps 6% of its images out of training and sends only their logits and labels, from which the server fits the temperatures; no image leaves a drone. The averaging in Eq. (1) is identical to Flower's FedAvg [12].

Round A trains the full model: a control with one drone holding all images, FedAvg with $K = 3$, 6 and 18 drones, local only training of the six drones, and two starts, for which the training towers are split into a server group (13 towers, 2,554 images) and a fleet group (5 towers, 2,553 images); fleet FedAvg starts either from the published checkpoint or from a model fine-tuned on the server group. Three further rounds vary one factor each:

- Update size, round B (6 tower drones, published checkpoint, 10 rounds): each drone trains and sends the decision head, the head with LoRA of rank 8 on every language layer [20] (merged into the base weights after training), the head with the last four or all 20 language layers, or the full model; further runs send the last-four update in int8 (one scale per tensor), add FedProx with $\mu = 0.01$ [11], or run 30 rounds.
- Few labels, round C (5 fleet drones, one tower each): each drone labels 25, 50, 100 or 200 images drawn at random from its tower (about 12% defects) and trains one local pass per round with the last-four update, from the published checkpoint or the server model; references are the server model, the same labels pooled centrally, and each drone alone.
- Image types, round D (6 drones, the 5,107 images, published checkpoint, last-four update): the images are split at random, by tower, by component, or with Dirichlet ($\alpha = 0.3$) condition shares. In the component split, corrosion (shackles only), missing caps (glass insulators only) and bird nests (vari-grip clamps only) each sit on one drone.

3.6. Evaluation

All numbers are on the test towers. For component and condition we report accuracy, balanced accuracy and macro F1, because the majority class covers 90% of the test images. For defect we report AUROC, average precision and expected calibration error (ECE). We tune a threshold on the validation towers that minimizes a risk of 10 per missed defect and 1 per false alarm, and report this risk per 1,000 images. Intervals are 95% bootstrap intervals over 1,000 resamples of the test images. Each configuration has one training seed.

Training sets with a different class mix than the test data shift the probabilities. As a correction that needs no retraining [18], we also report

$$\tilde{p}(c \mid x) \propto p(c \mid x) \, \frac{\pi_{\text{val}}(c)}{\pi_{\text{train}}(c)}, \tag{2}$$

where π are the label shares of the validation towers and of the training images. For comparison with earlier tools on the same benchmark, we also score the 7,057 test items of ElecVQA-Bench. A yes/no item takes $P(\text{defect}) \geq 0.5$, and a multiple-choice item takes the best condition among its options. The reference tools, evaluated on the same items, are a ResNet-50 [19] classifier fine-tuned on the training towers, SmolVLM-256M tuned with LoRA [20] as in EVLM, and the same SmolVLM without tuning.

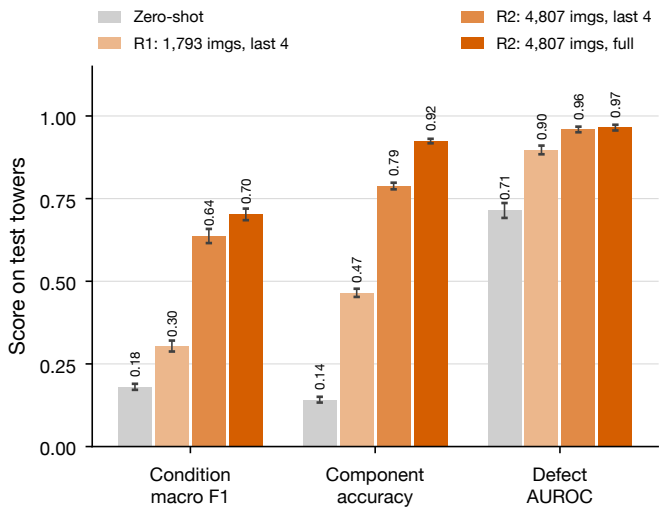


Figure 2: More data and a fully trained backbone lift every metric. Test-tower scores of laya-vision from zero-shot to the round-2 full fine-tune (6,092 images). Whiskers are 95% bootstrap intervals over images; one training seed per model; no class-share correction.

Table 1: Test-tower results. Images: 6,092 test images. Items: 7,057 ElecVQA-Bench test items. The intervals for laya-vision are in the text and in Fig. 2. Reference tools are scored on the same items. Bold: best per column.

Model	Cond. macro F1	Comp. acc.	Defect AUROC	Item acc.
Zero-shot	0.180	0.141	0.714	0.830
Round 1, last 4	0.303	0.465	0.898	0.702
Round 2, last 4	0.637	0.789	0.959	0.923
Round 2, full	0.703	0.924	0.965	0.948
ResNet-50, tuned	–	–	–	0.961
SmolVLM + LoRA	–	–	–	0.956
SmolVLM, no tuning	–	–	–	0.830

4. Results

4.1. Zero-shot

Without fine-tuning, laya-vision does not classify this data. Its condition macro F1 is 0.180 (95% interval 0.172–0.190), barely above the 0.158 of always answering good; it calls 480 of the 601 defect images good, and its component accuracy (0.141) is below the majority baseline (0.314). Only the defect question carries a weak signal (AUROC 0.714, ECE 0.178).

4.2. Fine-tuning

Figure 2 and Table 1 show the progression. Round 1 lifts the defect AUROC to 0.898, but it learned a mix with 79% defects and calls 1,971 of 5,491 good test images defective; the correction in Eq. (2) raises its macro F1 from 0.303 to 0.451. Round 2 improves every metric, and the full model is the best: condition accuracy 0.952 (above the majority answer), macro F1 0.703, component accuracy 0.924, defect AUROC 0.965 and ECE 0.058. For this model the correction changes little (condition accuracy 0.957).

Per condition, the full model has F1 0.975 for good, 0.710 for rust, 0.826 for corrosion, 0.762 for missing cap and 0.942 for bird nest; most errors are good images called rust (152), and the 5 test images of other defects are always missed. On the ElecVQA-Bench items it reaches 0.948, below the fine-tuned ResNet-50 (0.961) and SmolVLM with LoRA (0.956). Training takes 24 minutes on an RTX 2080 Ti.

4.3. Latency

On the RTX 2080 Ti in fp32, one predict call with all four questions takes a median of 72.0 ms per image (p95 98.5 ms, 74.0 ms after fine-tuning), including image loading; the condition question alone takes 71.4 ms, so further questions are nearly free. With 3.45 key frames per 4 s segment [2], a segment needs 0.25 s.

We have no Jetson measurement. A factor of 9, which we estimated between a GV100-class GPU and a Jetson Orin NX from fp32 throughput and memory bandwidth, gives about 0.67 s per image, about a quarter of the 2.8 s per caption of SmolVLM-256M on a Jetson Orin [1]. The comparison is approximate: the hardware differs, and the other systems generate text.

4.4. The decision model as a semantic trigger

Unlike a detector, the decision model judges the condition and can itself decide which frames a drone escalates. We use the defect threshold tuned on the validation towers (0.057). On the test towers it passes 931 of 6,092 images (15.3%) to a following stage and finds 532 of the 601 defects (recall 0.885), with a false-alarm rate of 0.073 among good images and a risk of 179 per 1,000 images, against 987 for always answering good. The cascade itself is not deployed.

4.5. Calibration and confidence

After fine-tuning, the defect probability has an ECE of 0.058 (0.178 zero-shot) and an average precision of 0.862. The judgeable query is no confidence signal (AUROC for a correct condition answer 0.53 zero-shot, 0.31 fine-tuned).

4.6. Federated fine-tuning across drones

The control with one drone matches central training: macro F1 0.697 against 0.703 (Table 2). The intervals overlap for all three metrics, so the training loop does not cause the differences below. With the same data spread over more drones, FedAvg loses accuracy (Fig. 3b, c). From 1 to 18 drones, the condition macro F1 falls from 0.697 to 0.474 and the component accuracy from 0.933 to 0.700. The defect AUROC degrades less, from 0.959 to 0.933, so the trigger of Section 4.4 is the most robust output under federation. The 6- and 18-drone runs are still improving after ten rounds (Fig. 3a), while the control has levelled off; Section 4.8 tests more rounds.

Federation still helps the individual drone. With six drones, FedAvg beats the average drone that trains alone

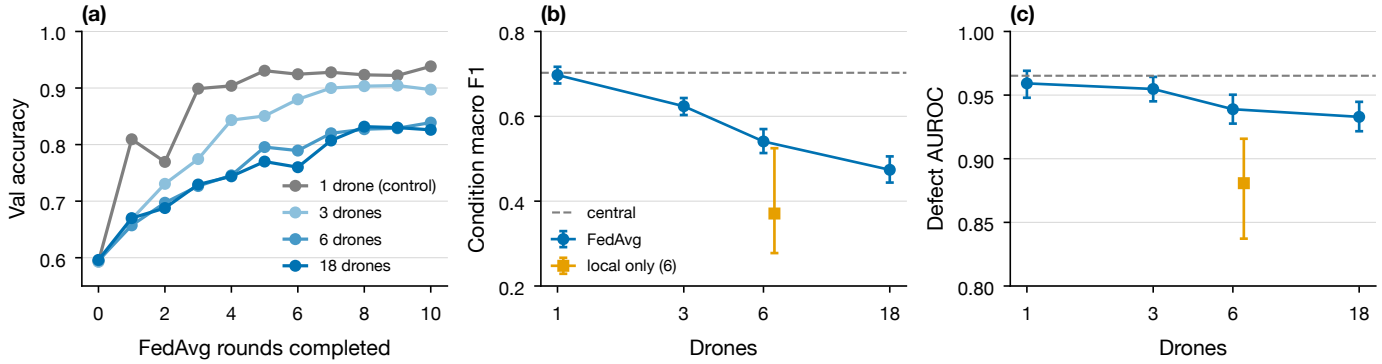


Figure 3: Splitting the same data over more drones slows FedAvg. (a) Validation accuracy on 600 images after each round. (b, c) Test-tower condition macro F1 and defect AUROC against the number of drones. FedAvg whiskers are 95% bootstrap intervals. The local-only marker is the mean of six drone models, and its whisker spans their minimum to maximum. The dashed line is central training. Every run uses the same total training (2.5 passes), with one seed per run.

on every metric (macro F1 0.541 against 0.371, defect AUROC 0.939 against 0.881) and the best single drone in defect AUROC (0.939 against 0.916); the best drone's macro F1 (0.525) lies inside the FedAvg interval.

4.7. A fine-tuned start for the fleet

On the same five fleet drones, the full-model FedAvg reaches a macro F1 of 0.513, a component accuracy of 0.593 and a defect AUROC of 0.932 from the published checkpoint, but 0.636, 0.912 and 0.953 from the server model (Table 2); the intervals of the two starts do not overlap. The fleet also improves the server model (0.558 to 0.636 macro F1, 0.903 to 0.953 AUROC) while its images stay on the drones, and its item accuracy (0.937) is close to central training on all towers (0.948).

4.8. What each drone sends

Each drone sends exactly the part it trains (sizes in Table 2); the vision encoder (86.4M parameters) is always frozen, and token embeddings make up a quarter of the full update.

Trained centrally, the same last-four update reaches 0.637 (Table 1), so the small update is not the problem in itself; under FedAvg its validation accuracy is still rising after 30 rounds (0.66 after 10, 0.81 after 30). Our reading, supported but not proven, is that averaging six small changes per round moves the shared model slowly, and an update with more parameters moves faster per step.

Uploading all 4,807 training images once would take only about 180 MB, so in this fleet federation keeps images on the drones rather than saving bandwidth.

4.9. Uplink against accuracy

Table 2 orders all 25 federated configurations by the total uplink of a run, from 1,014 to 41,310 MB, and marks the best value of each metric within four uplink tiers. The uplink alone does not predict accuracy.

Lowest (1,014–1,356 MB). Head, LoRA and the int8 last-four update from the published checkpoint stay close to the zero-shot model (macro F1 0.243–0.278); int8 is best in this tier and matches the fp16 last-four update (0.281) at half its uplink. Middle (2,260–2,712 MB). The same last-four update is useless from the published checkpoint (0.25–0.29 whatever the split or the labels) but almost as good as the full model from the server model: five fleet drones reach 0.565–0.582 with 25–200 labels each, and a defect AUROC of 0.930–0.933 with 100–200. This is the best accuracy per megabyte in the table: 2,260 MB, 20% of the uplink of the full-model fleet run (11,475 MB), give 92% of its macro F1. High (6,885–13,770 MB). The full model and all language layers learn within ten rounds (0.488–0.636); the best macro F1 and component accuracy (0.636, 0.912) again come from the server-model start. Highest (41,310 MB). 18 drones triple the uplink of six drones but lower the macro F1 from 0.541 to 0.474. For a given uplink, the starting point thus matters more than the update size; round B suggests that int8 would halve the 45 MB update again, which remains to be confirmed from the fine-tuned start.

4.10. Few labels per drone

A real fleet labels few images, mostly of good components: 25, 50, 100 and 200 labels per drone contain only 18, 37, 69 and 149 defects across the fleet. From the published checkpoint they achieve almost nothing (macro F1 0.159–0.207, Fig. 4). Starting from the server model, the macro F1 barely moves, but the defect AUROC follows a U-shape: it first falls below the server model, to 0.856 with 18 labelled defects and 0.836 with 37, and then rises above it, to 0.930 with 69 and 0.933 with 149.

Table 3 explains the dip. With few defects, federated averaging makes the model conservative: with 18 labelled defects, the false-alarm rate falls from 0.139 to 0.043 but the defect recall falls from 0.832 to 0.740. The cause is the averaging, not the number of labels. Pooling the same 37 defects on one server keeps the AUROC at the level of the server model (0.904) and lowers the risk to 211, whereas FedAvg over drones that hold between 0 and 15

Table 2: All federated configurations, ordered from the lowest to the highest total uplink (test towers, one seed per run). Start: published checkpoint (base) or server model (tuned). Update: size of one drone update in MB (fp16 unless int8; parameter counts in brackets). Uplink: update size × drones × rounds in MB, calculated. Bold: best value of a metric within its uplink tier. [a]Upload of all 4,807 training images once, no model update.

Configuration	Start	Trained part	Update	Drones×rounds	Uplink (MB)	Macro F1	Comp. acc.	AUROC
References (no model updates or one drone)								
Central, all 18 towers	–	–	–	1×1	180[a]	0.703	0.924	0.965
Control, 1 drone	base	full	229.5	1×10	2,295	0.697	0.933	0.959
Server towers only	base	full	–	–	0	0.558	0.894	0.903
Local only, 6 drones (mean)	base	full	–	–	0	0.371	0.621	0.881
Lowest uplink: 1,014–1,356 MB								
Head (8.5M)	base	head	16.9	6×10	1,014	0.243	0.293	0.792
Head + LoRA r8 (10.1M)	base	LoRA	20.2	6×10	1,212	0.247	0.307	0.828
Last 4 + head, int8 (22.6M)	base	last 4	22.6	6×10	1,356	0.278	0.320	0.838
Middle: 2,260–2,712 MB								
Few labels, 25/drone	base	last 4	45.2	5×10	2,260	0.159	0.196	0.721
Few labels, 50/drone	base	last 4	45.2	5×10	2,260	0.159	0.371	0.726
Few labels, 100/drone	base	last 4	45.2	5×10	2,260	0.162	0.441	0.767
Few labels, 200/drone	base	last 4	45.2	5×10	2,260	0.207	0.509	0.838
Few labels, 25/drone	tuned	last 4	45.2	5×10	2,260	0.565	0.898	0.856
Few labels, 50/drone	tuned	last 4	45.2	5×10	2,260	0.571	0.903	0.836
Few labels, 100/drone	tuned	last 4	45.2	5×10	2,260	0.582	0.912	0.930
Few labels, 200/drone	tuned	last 4	45.2	5×10	2,260	0.575	0.914	0.933
Last 4 + head, tower split	base	last 4	45.2	6×10	2,712	0.281	0.321	0.838
+ FedProx 0.01	base	last 4	45.2	6×10	2,712	0.275	0.331	0.834
random split	base	last 4	45.2	6×10	2,712	0.254	0.334	0.856
component split	base	last 4	45.2	6×10	2,712	0.287	0.331	0.837
component + FedProx	base	last 4	45.2	6×10	2,712	0.285	0.339	0.837
Dirichlet split	base	last 4	45.2	6×10	2,712	0.291	0.477	0.833
High: 6,885–13,770 MB								
Full model, 3 drones	base	full	229.5	3×10	6,885	0.624	0.874	0.955
Last 4 + head, 30 rounds	base	last 4	45.2	6×30	8,136	0.488	0.634	0.945
All 20 language layers (79.3M)	base	layers	158.5	6×10	9,510	0.488	0.707	0.931
Full model, 5 fleet drones	base	full	229.5	5×10	11,475	0.513	0.593	0.932
Full model, 5 fleet drones	tuned	full	229.5	5×10	11,475	0.636	0.912	0.953
Full model (114.7M), 6 drones	base	full	229.5	6×10	13,770	0.541	0.761	0.939
+ FedProx 0.01	base	full	229.5	6×10	13,770	0.526	0.756	0.940
Highest: 41,310 MB								
Full model, 18 drones	base	full	229.5	18×10	41,310	0.474	0.700	0.933

defects each lowers the AUROC to 0.836. With 149 defects, pooling is still better (AUROC 0.958 against 0.933), but FedAvg beats the average drone alone (0.902). In this fleet, federated training pays off only once the drones have labelled several defects, about 70 across the fleet; where the labels may be pooled, pooling is better and needs far less uplink (about 37 MB of images against 2,260 MB of updates).

4.11. Drones with different image types

The Jensen–Shannon divergence between the condition shares of a drone and of the fleet ranges from at most 0.002 (random) over 0.009–0.070 (tower) and 0.128–0.333 (component) to 0.090–0.665 (Dirichlet). Although the splits differ this strongly, they give the same macro F1 within their 95% intervals (0.254–0.291 at 2,712 MB each, Table 2), and FedProx changes nothing. Figure 5 shows why. Each drone that trains alone learns the defect type of its own components well (corrosion 0.86, bird nest 0.94, missing cap 0.79), but FedAvg with the last-four update loses all three types, and it loses them just as much under the random split, where every drone sees every type. The limiting factor is therefore the slow small update, not the differences between drones, and with this update the experiment cannot measure the effect of the split. Full-model FedAvg keeps most of them. A specialist drone, in turn, barely recognizes the other drones' components (accuracy 0.16).

5. Discussion

Perception for task planning. A large part of the inspection decision does not need a description. From one image encoding, the decision model answers component, defect and condition queries with accuracy close to a fine-tuned describing model, needs no synthetic captions and gives calibrated probabilities that a planner can threshold. At 74 ms per image it leaves time on board for the planner itself: the defect probability can decide which frames are escalated (15.3% at 88.5% recall), and the component and condition answers can mark assets for a revisit. A describ-

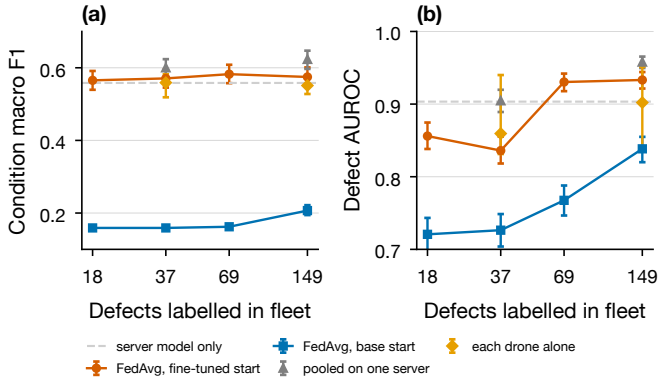


Figure 4: Fleet labels help only from the fine-tuned start, and only once the fleet has labelled enough defects. Five fleet drones with 25, 50, 100 or 200 natural labels each (about 12% defects), plotted against the number of defects labelled across the fleet; last-four update, 10 rounds. Whiskers: 95% bootstrap intervals; for drones alone, the range of the five drone models. The dashed line is the server model without fleet labels. One seed per run.

Table 3: Defect detection with few labels (test towers). Risk, recall and false-alarm rate use the threshold tuned on the validation towers; risk counts 10 per missed defect and 1 per false alarm per 1,000 images; FAR is the false-alarm rate. Uplink: total MB of the run; for pooling, the labelled images (about 37 kB each, estimated from the mean file size). Drones alone: mean of the five drone models. Bold: best per column. One seed per run.

Model	Defects	Uplink	AUROC	Risk	Recall	FAR
Server model only	–	0	0.903	291	0.832	0.139
FedAvg, tuned	18	2,260	0.856	295	0.740	0.043
	37	2,260	0.836	331	0.684	0.021
	69	2,260	0.930	332	0.900	0.259
	149	2,260	0.933	280	0.854	0.150
FedAvg, base	18	2,260	0.721	687	0.809	0.553
	149	2,260	0.838	464	0.844	0.344
Pooled	37	≈9	0.904	211	0.832	0.050
	149	≈37	0.958	181	0.870	0.059
Drones alone	37	0	0.860	319	0.730	0.059
	149	0	0.902	272	0.812	0.096

ing VLM remains useful for reports and for defects outside the option set (Fig. 1).

Distributed intelligence across the fleet. FedAvg across tower-wise drones beats training alone, and a server-fine-tuned start brings the fleet close to central training (Table 2). Its costs are accuracy as the data is split further and, for the full model, 6,885–41,310 MB of uplink per run. For the fleet operator, our results suggest a training plan: fine-tune centrally on the data the operator may hold, let the fleet continue with a small update (45 MB per drone and round, which int8 can likely halve), and start a federated round only when the fleet has labelled enough defects and the labels cannot be pooled. Which drones join a round, and with which update, is thus itself a planning decision that trades accuracy against uplink.

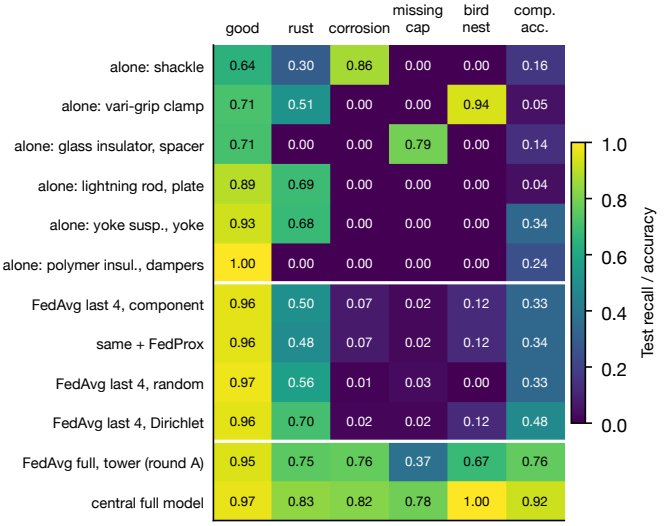


Figure 5: Specialist drones learn their own defect type; small-update FedAvg keeps none of them. Test recall per condition and component accuracy (last column). Top: each drone of the component split trained alone (corrosion exists only on shackles, bird nests only on vari-grip clamps, missing caps only on glass insulators). Middle: FedAvg with the last-four update from the published checkpoint under four splits. Bottom: full-model FedAvg of round A and central training. One seed per model.

Limitations. All configurations have one seed, the drones are simulated on one GPU (the uplink is calculated), the Jetson latency is an estimate, and energy and the planner itself are not measured. The data cover one benchmark and one asset type. The update-size and image-type runs start from the published checkpoint, where small updates learn slowly; repeating them from the fine-tuned start with int8, with more seeds and a Jetson measurement, is the next step.

6. Conclusion

A 201M-parameter vision-language decision model fails zero-shot on power-line component images, but after fine-tuning on folder labels it reaches 0.948 item accuracy at 74 ms per image for four queries, with calibrated defect probabilities that can drive the escalation decisions of an inspection drone. Trained across a simulated fleet, the starting point matters more than the uplink: a server-fine-tuned start lets a 45 MB update reach 92% of the full-model fleet result with 2,260 instead of 11,475 MB of uplink, whereas from the published checkpoint small updates barely learn. With few labels, the fleet helps only once it has labelled enough defects, and pooling the same labels remains better.

Data and code availability

The code, the per-image predictions and the evaluation files are in the project repository, branch vision-fl (to

be made public on acceptance). ElecVQA-Bench and InsPLAD are public [16, 17]. The laya-vision weights are published under CC BY-NC-SA 4.0 [6].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text. The author reviewed and edited the content and takes full responsibility for the publication.

Acknowledgments

Experiments in this work were run and tracked with OpenResearch [21].

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[5] Convai Innovations, Laya, software (2026). https://github.com/NandhaKishorM/laya.

[6] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[7] Sabanovic, Maliakel, Brandic, INAR-VL, arXiv:2605.18853 (2026).

[8] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), 2017.

[9] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.

[10] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-IID data, arXiv:1806.00582 (2018).

[11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), 2020.

[12] D. J. Beutel, et al., Flower: A friendly federated learning research framework, arXiv:2007.14390 (2020).

[13] Askin, et al., Federate the router, arXiv:2601.22318 (2026).

[14] Solat, Lee, Seif, Niyato, Poor, FedHLM, IEEE Internet of Things Journal (2025). https://doi.org/10.1109/jiot.2025.3618095.

[15] From open-vocabulary detection to structural assessment: An agentic federated approach for guardrail monitoring (2026). https://www.sciencedirect.com/science/article/pii/S2772503026000459.

[16] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[17] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[18] M. Saerens, P. Latinne, C. Decaestecker, Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure, Neural Computation 14 (1) (2002) 21–41.

[19] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[20] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[21] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.